

ORACLE-BASED MULTISTEP STRATEGY FOR SOLVING POLYNOMIAL SYSTEMS OVER FINITE FIELDS AND ALGEBRAIC CRYPTANALYSIS OF THE ARADI CIPHER

ROBERTO LA SCALA* AND SHARWAN K. TIWARI**

ABSTRACT. The multistep solving strategy consists in a divide-and-conquer approach: when a multivariate polynomial system is computationally infeasible to solve directly, one variable is assigned over the elements of the base finite field, and the procedure is recursively applied to the resulting simplified systems. In a previous work by the same authors (among others), this approach proved effective in the algebraic cryptanalysis of the TRIVIUM cipher.

In this paper, we present a new formulation of the corresponding algorithm based on a Depth-First Search strategy, along with a novel complexity analysis leveraging tree structures. We also introduce the notion of an “oracle function”, which is intended to determine whether evaluating a new variable is required to simplify the current polynomial system. This notion allows us to unify all previously proposed variants of the multistep strategy, including the classical hybrid approach, by appropriately selecting the oracle function.

Finally, we employ the multistep solving strategy in the cryptanalysis of the NSA’s recently introduced low-latency block cipher ARADI, achieving a first full-round algebraic attack that exposes structural features in its symbolic model.

1. INTRODUCTION

Among NP-hard problems, one with immediate and wide-ranging applications is the so-called MP-problem, which involves solving a system of multivariate polynomial equations over a finite field $\mathbb{F} = \text{GF}(q)$. When $q = 2$, this problem corresponds to the SAT-problem, and many challenges in cryptography are directly related to the MP-problem, including algebraic cryptanalysis and multivariate public-key cryptography.

Among the methods used to solve the MP-problem are symbolic algorithms, such as Gröbner bases and XL linearization [7, 8, 9], SAT solvers [17] for the Boolean case and discrete optimization techniques like Quantum Annealing [18]. Each of these methods can be advantageous in specific cases, but in general, they all exhibit a complexity comparable to the exponential complexity q^n of the brute-force approach, with n representing the number of variables in the polynomial system.

2000 *Mathematics Subject Classification.* Primary 13P15. Secondary 11T71, 11T06.

Key words and phrases. Polynomial system solving; Finite fields; Cryptanalysis.

The first author acknowledges the partial support of PNRR MUR projects “Security and Rights in the CyberSpace”, Grant ref. CUP H93C22000620001, Code PE00000014, Spoke 3, and “National Center for HPC, Big Data and Quantum Computing”, Grant ref. CUP H93C22000450007, Code CN00000013, Spoke 10. The same author was co-funded by MUR project PRIN 2022SC, Grant ref. 2022RFAZCJ, and Università di Bari “Fondo acquisto e manutenzione attrezzature per la ricerca”, Grant ref. DR 3191.

Beyond theoretical worst-case complexity, valid when the polynomials in the system are randomly chosen, it is especially important in cryptanalysis to estimate the actual computational cost of solving a specific polynomial system over a finite field through experimentation. Since solving such systems becomes infeasible when the number of variables is sufficiently large, a standard approach is to combine brute force with polynomial solving in what is known as a “hybrid strategy”. Specifically, this approach assumes that one can feasibly solve all derived systems obtained by exhaustively assigning values in the finite field to k out of the n variables in the original system. By using a test set of such partial evaluations, it is possible to estimate the average solving time τ for a single derived system. Consequently, the total expected complexity of the hybrid approach becomes $q^k \tau$.

One issue with this approach is that, in order to ensure the actual solvability of all derived systems, the number k of evaluated variables may become quite large. To address this, the paper [14] proposes a dynamic approach in which the number of evaluated variables is adjusted based on the specific evaluation. In other words, starting from an initial evaluation $x_1 = c_1, \dots, x_k = c_k$ ($c_1, \dots, c_k \in \mathbb{F}$), a set of q additional assignments $x_{k+1} = c_{k+1}$, for all $c_{k+1} \in \mathbb{F}$, is considered, but only when the polynomial system associated with the initial evaluation exhibits features that indicate it may be difficult to solve. This divide-and-conquer strategy is referred to as a “multistep strategy”.

In [14], the key feature used to predict the actual solvability of the system was the number of variables remaining after a preprocessing step, which was required to be below a given threshold. This step aimed to generate additional linear polynomials in the ideal, beyond those corresponding to variable assignments, in order to eliminate as many variables as possible. The preprocessing was based on the computation of a truncated Gröbner basis up to a low degree.

Since the number of remaining variables does not yield accurate predictions in the case of sparse polynomial systems encountered in the algebraic cryptanalysis of certain block ciphers, in this paper we introduce an abstract notion of an “oracle function”. This function determines whether the multistep strategy should attempt to solve the current system or instead perform a branching operation by assigning a new variable across the elements of the finite field. The recursive calls in a Depth-First Search implementation of the multistep strategy naturally define a tree structure, whose analysis provides insights into the algorithm’s complexity. The concept of oracle function provides a unified framework that generalizes different solving strategies, including the one used in [14] and the classic hybrid approach, into a single algorithm in which only the oracle function varies. Furthermore, it is plausible that machine learning techniques may aid in the construction of effective oracle functions tailored to specific problem settings. Finally, this paper demonstrates the applicability of the multistep solving strategy through a full-round algebraic attack on a modern block cipher.

Low-latency block ciphers have gained prominence in applications such as secure memory encryption, where high throughput and minimal critical path delay are essential. ARADI, a 128-bit block cipher with a 256-bit key, was recently proposed in [11] as a candidate to meet these demands. ARADI employs a substitution-permutation network (SPN) architecture composed of word-wise Toffoli gates, word permutations, and lightweight linear maps. Despite its promising efficiency, the

original specification offers limited justification for the cipher’s structural design and includes only a minimal security analysis.

Following the public release of ARADI and its authenticated encryption mode LLAMA by NSA researchers Greene, Motley, and Weeks [11], several studies have explored its cryptographic strength and structural properties. One of the earliest responses came from Avanzi, Dunkelman, and Ghosh [1], who questioned ARADI’s practical advantage over existing lightweight ciphers in terms of area-latency trade-offs. They also identified issues in the LLAMA mode related to the use of variable-length initialization vectors, which compromised the integrity and confidentiality of ciphertexts. These issues were later acknowledged by the designers in an updated ePrint version.

The first comprehensive cryptanalytic evaluation was conducted by Bellini et al. [6] using the CLAASP framework for automated trail-based analysis. Their study includes statistical black-box testing (such as avalanche and diffusion analysis), the discovery of differential and linear trails up to 9 and 10 rounds, impossible differentials up to 8 rounds, and neural distinguishers up to 5 rounds. A preliminary algebraic analysis is also presented, revealing an integral distinguisher for 7 rounds requiring 2^{124} data, along with an 11-round key-recovery attack based on a 9-round differential trail, with data and time complexities of 2^{126} and 2^{252} , respectively. These results indicate slow diffusion and weak mixing in the early rounds of ARADI.

Another study by Bellini et al. [5] focuses on the algebraic structure of the Toffoli-based S-box layer in ARADI. The authors identify a structural property that allows any r -round integral distinguisher to be extended to $(r + 1)$ -rounds without increasing data complexity, based on the predictable interaction of bitwise Toffoli gates. They show that this phenomenon is preserved under various changes to the linear layer and Toffoli gate ordering. The paper presents 8-round algebraic distinguishers and a 10-round key-recovery attack requiring 2^{124} data and 2^{177} time.

The structural property was first analyzed in a preliminary ePrint report [4], where MILP-based modeling of the three-subset division property without unknown subsets was applied to ARADI to derive degree bounds. Building on this foundation, a follow-up study [12] extended the analysis to 8 rounds by refining the MILP constraint formulation introduced in [4]. This extension further confirmed the persistence of the structural weakness across various constraint encoding strategies.

In this work, we study the cryptanalytic properties of ARADI using a symbolic algebraic approach. Specifically, we derive an implicit representation of the cipher’s round function as a multivariate polynomial map over the Boolean field $\mathbb{F} = \text{GF}(2)$. This includes a full symbolic description of key addition, the nonlinear S-box layer, and the word-wise linear transformation, as defined in the cipher’s specifications [11]. Using this symbolic model, we instantiate the *full 16-round* cipher with a known plaintext–ciphertext pair to obtain a multivariate polynomial system over $\mathbb{F}$, whose unknowns correspond to the 256 key bits and the internal variables introduced by the S-box layers.

To solve this polynomial system over the base field $\mathbb{F}$, we employ a multistep strategy that explores the cipher’s algebraic structure. Specifically, it recursively considers partial key assignments using a Depth-First Search approach. The approach integrates a timeout-based Gröbner basis subroutine, used to probe whether the resulting polynomial system is feasibly solvable.

We conduct comprehensive experiments to evaluate the efficiency and reliability of the proposed method. Specifically, we compare the multistep strategy with the classical hybrid approach by measuring the success rate of computing a complete Gröbner basis under a fixed solving degree and within a predefined time limit, across a test set involving the evaluation of different numbers of variables. Our results show that in approximately 99.98% of cases, the polynomial systems can be efficiently solved after guessing just 251 out of 256 key bits. For the remaining 0.02%, the multistep strategy automatically switches to 252 evaluations. Consequently, this strategy achieves a somewhat higher efficiency than the classic hybrid method, which requires 252 guesses to ensure that 100% of systems can be solved efficiently.

Moreover, we observe that all derived systems are quadratic and can be solved with a solving degree of two when more than 250 key variable assignments are provided. However, this is no longer possible when the number of assignments is reduced to 250. These findings point to some thresholds within ARADI's algebraic structure, and they show that a dynamic multistep strategy can be practically useful in algebraic cryptanalysis.

2. MULTISTEP ALGORITHM

Let $\mathbb{F} = \text{GF}(q)$ be a finite field and consider the multivariate polynomial algebra $R = \mathbb{F}[x_1, \dots, x_n]$ and the field equation ideal $\mathcal{F} = \langle x_1^q - x_1, \dots, x_n^q - x_n \rangle \subset R$. Let $J = \langle f_1, \dots, f_m \rangle \subset R$ be an ideal such that $\#V_{\mathbb{F}}(J) \leq 1$. Put

$$H = \{f_1, \dots, f_m\} \cup \{x_1^q - x_1, \dots, x_n^q - x_n\}$$

that is, H is a generating set of the ideal $J + \mathcal{F}$. To simplify notations, we denote $\mathbb{F} = \{c_1, \dots, c_q\}$. Since $\#V_{\mathbb{F}}(J) \leq 1$, we have that a (reduced) Gröbner basis of $J + \mathcal{F}$ with respect to any monomial ordering of R , is either $G = \{1\}$ or $G = \{x_1 - c_{i_1}, \dots, x_n - c_{i_n}\}$, for some $1 \leq i_1, \dots, i_n \leq q$ (see, for instance, [10, 13, 15]). In other words, $\maxdeg(G) = \max\{\deg(f) \mid f \in G\} \leq 1$.

Let $d_1 \geq 0$ be an integer and denote by $\text{GROEBNER}(H, d_1)$ the algorithm performing an incomplete Gröbner basis computation which is stopped when all S-polynomials of degree $\leq d_1$ have been considered at a current step. This algorithm corresponds having a bound on the size of the Macaulay matrix when using linear algebra methods for computing the Gröbner basis. If the degree d_1 is sufficiently large, we have that $G = \text{GROEBNER}(H, d_1)$ is a complete Gröbner basis of $J + \mathcal{F}$. In this case, we say that d_1 is a *solving degree* for the ideal $J + \mathcal{F}$ with respect to the given monomial ordering. However, if $d_1 < \maxdeg(H)$, then $G = \text{GROEBNER}(H, d_1)$ is generally not a generating set of the ideal $J + \mathcal{F}$. Instead, a generating set is given by $G \cup G'$, where $G' = \{g \in H \mid \deg(g) > d_1\}$.

A useful procedure within the solving algorithm is GBELIMLIN. Its goal is to generate linear polynomials belonging to the ideal $J + \mathcal{F}$ in order to eliminate variables from its generators. The method consists of computing an incomplete Gröbner basis up to a sufficiently low degree d_1 to keep the computation efficient. The expectation that linear polynomials appear among the generators arises from the fact that the complete Gröbner basis is entirely linear when $\#V_{\mathbb{F}}(J) = 1$. Note also that, when incorporating this procedure into the multistep solving algorithm, we explicitly add linear polynomials corresponding to variable evaluations to the generating set.

Algorithm 2.1 GBELIMLIN

Input: a generating set H of the ideal $J + \mathcal{F}$ and an integer $d_1 \geq 0$;
Output: two subsets $L, G \subset J + \mathcal{F}$ where $\deg(g) = 1$ for all $g \in L$ and
 $\langle G \rangle = (J + \mathcal{F}) \cap R'$ with R' the polynomial algebra in the variables
occurring in G .

$G := \{g \in H \mid \deg(g) \leq d_1\};$
 $G' := H \setminus G;$
 $G := \text{GROEBNER}(G, d_1);$
 $G := G \cup G';$
if $\max\deg(G) \leq 1$ **then**
 return $\emptyset, G;$
end if;
 $L := \{g \in G \mid \deg(g) = 1\};$
 $G' := G \setminus L;$
 $S := \{x_i^q - x_i \mid x_i \text{ occurs in } G'\};$
 $G := \text{REDUCE}(G', L \cup S);$
 $S := \{x_i^q - x_i \mid x_i \text{ occurs in } G'\};$
 $G := G \cup S;$
return $L, G;$

In the GBELIMLIN algorithm, we assume that the linear polynomials in the set L are fully interreduced, meaning they are in reduced row echelon form in the Macaulay matrix used to compute $\text{GROEBNER}(G, d_1)$. The REDUCE procedure implements the full reduction algorithm from Gröbner basis theory. Note that if $\max\deg(G) \leq 1$, then G is a Gröbner basis of $J + \mathcal{F}$. In other words, GBELIMLIN directly returns $V_{\mathbb{F}}(J)$, as d_1 is a solving degree in this case. Finally, note that a modified version of GBELIMLIN could be iteratively applied to enhance its capability to eliminate variables.

To describe our solving algorithm, it is also helpful to consider a variant of GROEBNER with a timeout τ to limit the computation time. Specifically, we define the following procedure.

Algorithm 2.2 GROEBNERSAFE

Input: a generating set H of an ideal $J + \mathcal{F}$, an integer $d_2 \geq \max\deg(H)$ and a real number $\tau > 0$;
Output: either **wild**, $\emptyset$ or **tame**, G where $G = \text{GROEBNER}(H, d_2)$ and $\max\deg(G) \leq 1$;
compute $G := \text{GROEBNER}(H, d_2)$ within timeout τ ;
if computation completes within τ and $\max\deg(G) \leq 1$ **then**
 return **tame**, G ;
end if
return **wild**, $\emptyset$;

We finally present the solving algorithm MULTISOLVE. To describe it, we need to introduce the following subsets of R . For all $1 \leq i_1, \dots, i_k \leq q$ and $0 \leq k \leq n$, we put

$$E(i_1, \dots, i_k) = \{x_1 - c_{i_1}, \dots, x_k - c_{i_k}\}.$$

For $k = 0$, we put $E() = \emptyset$ by definition. Adding the subset $E(i_1, \dots, i_k)$ to the generators of the ideal J corresponds to evaluating the variables $x_1, \dots, x_k$ at the elements $c_{i_1}, \dots, c_{i_k} \in \mathbb{F}$. More precisely, we define the ideal

$$J(i_1, \dots, i_k) = \langle E(i_1, \dots, i_k) \rangle + J.$$

If $V_{\mathbb{F}}(J) = \{(c_{i_1}, \dots, c_{i_n})\}$, we have that

$$V_{\mathbb{F}}(J(j_1, \dots, j_k)) = \begin{cases} V_{\mathbb{F}}(J) & \text{if } (j_1, \dots, j_k) = (i_1, \dots, i_k); \\ \emptyset & \text{otherwise.} \end{cases}$$

The MULTISOLVE algorithm essentially follows a divide-and-conquer approach. If computing $V_{\mathbb{F}}(J(i_1, \dots, i_k))$ within a given time limit is not feasible, the algorithm instead attempts to compute $V_{\mathbb{F}}(J(i_1, \dots, i_k, i))$, for all $1 \leq i \leq q$. Indeed, we have

$$V_{\mathbb{F}}(J(i_1, \dots, i_k)) = \bigcup_{1 \leq i \leq q} V_{\mathbb{F}}(J(i_1, \dots, i_k, i))$$

Moreover, the generators of $J(i_1, \dots, i_k, i)$ involve fewer unknowns than those of $J(i_1, \dots, i_k)$. In other words, this approach simplifies the solving problem at the cost of branching over the elements of the base finite field.

The algorithm MULTISOLVE is implemented here as a recursive procedure of type Depth-First Search (DFS), which is alternative to the iterative algorithm of type Breadth-First Search (BFS) that was proposed in the paper [14].

Let H be a generating set of the ideal $J + \mathcal{F}$ such that $\#V_{\mathbb{F}}(J) \leq 1$. Let ORACLE be a function that takes as input a set of polynomials and an integer k , and returns either the string `tame` or the string `wild`. Finally, fix two integers $d_1 \geq 0$ and $d_2 \geq \maxdeg(H)$. The set H , along with the function ORACLE and the integers d_1, d_2 , are global variables for the following algorithm MULTISOLVE which applies the multistep solving strategy to the ideal $J(i_1, \dots, i_k)$. For $k = 0$, we have $J() = J$ by definition.

Algorithm 2.3 MULTISOLVE

Input: a k -tuple $(i_1, \dots, i_k)$ where $1 \leq i_1, \dots, i_k \leq q$ and $0 \leq k \leq n$;
Output: a Gröbner basis G of the ideal $J(i_1, \dots, i_k) + \mathcal{F}$;
 $G := E(i_1, \dots, i_k) \cup H$;
 $L, G := \text{GBELIMLIN}(G, d_1)$;
if $\text{maxdeg}(G) = 1$ **then**
 return $L \cup G$;
end if
if $\text{maxdeg}(G) = 0$ **then**
 return $\{1\}$;
end if
if $\text{ORACLE}(G, k) = \text{tame}$ **then**
 $\text{status}, G := \text{GROEBNERSAFE}(G, d_2, \tau)$;
 if $\text{status} = \text{tame}$ and $\text{maxdeg}(G) = 1$ **then**
 return $L \cup G$;
 end if
 if $\text{status} = \text{tame}$ and $\text{maxdeg}(G) = 0$ **then**
 return $\{1\}$;
 end if
end if
for all $i \in \{1, 2, \dots, q\}$ **do**
 $G' := \text{MULTISOLVE}(i_1, \dots, i_k, i)$;
 if $\text{maxdeg}(G') = 1$ **then**
 return G' ;
 end if
end for
return $\{1\}$;

Besides the DFS implementation, a key difference of the above algorithm from the one presented in [14] is the use of a general oracle function meant to predict when $\text{GROEBNERSAFE}(G, d_2, \tau)$ would return **status** = **wild**. In the previous version of MULTISOLVE, given an integer $B > 0$, the oracle function was simply defined as

$$\text{ORACLENRV}(G, k) = \begin{cases} \text{wild} & \text{if the number of remaining variables in } G \text{ is } > B; \\ \text{tame} & \text{otherwise.} \end{cases}$$

By remaining variables above, we refer to the variables that still occur in G after applying the GBELIMLIN procedure. Indeed, an accurate oracle predicting wild cases could eliminate unnecessary and time-consuming computations, significantly enhancing the efficiency of the algorithm. Although the number of variables plays a crucial role in determining the complexity of solving a polynomial system, many other features can also influence it, especially in the sparse case.

The termination of the MULTISOLVE algorithm follows directly from the fact that computing a Gröbner basis for the ideal $J(i_1, \dots, i_n) + \mathcal{F}$ is trivial. This is because all variables in R are explicitly evaluated through the set

$$E(i_1, \dots, i_n) = \{x_1 - c_{i_1}, \dots, x_n - c_{i_n}\}.$$

Of course, termination typically occurs well before evaluating all variables.

We remark that the MULTISOLVE algorithm essentially imposes a fixed solving degree d_2 , at the cost of performing branching among the various derived polynomial systems. Combined with its recursive structure, this provides MULTISOLVE with considerable simplicity and flexibility, which can be adjusted by varying the parameters d_1, d_2, τ and the function ORACLE.

3. COMPLEXITY

Let us analyze the complexity of MULTISOLVE when computing $V_{\mathbb{F}}(J(i_1, \dots, i_k))$. Recall that for $k = 0$, we have $J() = J$. Indeed, we assume that the computation time of GBELIMLIN is negligible, due to the low value of the parameter d_1 . Likewise, we assume that invoking the ORACLE function is negligible, as these functions merely involve comparing numerical data or performing a forward pass of a neural network, operations that are typically very fast.

As a result, the execution time of MULTISOLVE is the sum of the time spent by GROEBNERSAFE handling wild and tame cases when ORACLE = **tame**. We refer to cases corresponding to the status returned by GROEBNERSAFE as *computed wild* and *computed tame* cases. Similarly, cases determined by the values of the function ORACLE are referred to as *predicted wild* and *predicted tame* cases.

By a *case* or a *guess*, we mean here a l -tuple $(j_1, \dots, j_l)$ ($1 \leq j_1, \dots, j_l \leq q$, $k \leq l \leq n$) which defines the evaluation set

$$E(j_1, \dots, j_l) = \{x_1 - c_{j_1}, \dots, x_l - c_{j_l}\}.$$

These guesses arise during the execution of MULTISOLVE and correspond to its recursive calls. The following two notions are useful for the complexity analysis of MULTISOLVE.

Definition 3.1. *The function ORACLE is called accurate if no computed wild cases occur during the execution of MULTISOLVE. It is called perfect if, in addition, it minimizes the number of computed tame cases. In other words, an oracle is accurate if every predicted tame case corresponds to a computed tame case, and it is perfect if the prediction and computation always agree.*

By assuming that the oracle function is accurate, the complexity of MULTISOLVE is reduced to the number of computed tame cases. In other words, there are no computed wild cases incurring the timeout cost τ , and consequently no significant overhead arises in dealing with wild and tame cases according to the multistep solving strategy. Indeed, the oracle function associated with the MULTISOLVE algorithm in [14] is assumed to be accurate. Moreover, given an integer $B > 0$, the standard hybrid strategy, as described in [2, 3], can be obtained as the MULTISOLVE algorithm implementing an accurate oracle function of type

$$\text{ORACLEH}_B(G, k) = \begin{cases} \text{wild} & \text{if } k < B; \\ \text{tame} & \text{otherwise.} \end{cases}$$

Another useful concept is the following.

Definition 3.2. *A q -ary tree is by definition a tree in which each node has either zero or exactly q children. In other words, every internal node has q children, while leaf nodes have no children.*

Note that the recursive calls of the MULTISOLVE algorithm form a q -tree, where each node represents a guess $(j_1, \dots, j_l)$. Internal nodes correspond to wild cases,

whether predicted or computed, while the leaves represent the computed tame cases. More precisely, we have a complete q -tree when $V_{\mathbb{F}}(J(i_1, \dots, i_k)) = \emptyset$. Conversely, when $V_{\mathbb{F}}(J(i_1, \dots, i_k)) \neq \emptyset$, the tree consists of only a subset of nodes, as recursive calls are terminated as soon as the solution is found. We have the following useful result concerning q -trees.

Theorem 3.3. *In a q -ary tree, let N be the total number of nodes, M the number of internal nodes and L the number of leaves. Then, the following equations holds*

$$N = \frac{qL - 1}{q - 1}, M = \frac{L - 1}{q - 1}.$$

In particular, when $q = 2$, we have $N = 2L - 1$ and $M = L - 1$.

Proof. We prove the first formula by induction on the number of leaves L . When there is only one leaf, the tree consists of a single node. In this case, we have $N = L = 1$, which satisfies the formula.

Assume that the first formula holds for any q -ary tree with strictly fewer than L leaves. Consider a q -ary tree T having N nodes and L leaves. This tree can be obtained by a smaller tree T' by converting one of its leaves into an internal node and attaching q new leaves to it. Let N' and L' be the number of nodes and leaves of T' , respectively. Then, we have

$$N = N' + q, L = (L' - 1) + q.$$

Since $L > L'$, we can apply the induction hypothesis to T' , yielding

$$N = \frac{qL' - 1}{q - 1} + q = \frac{q(L + 1 - q) - 1}{q - 1} + q = \frac{qL - 1}{q - 1}.$$

The second formula is obtained directly from the first by subtracting L . □

Definition 3.4. *During the execution of the algorithm $\text{MULTISOLVE}(i_1, \dots, i_k)$, we call depth the current number of evaluated variables in addition to $x_1, \dots, x_k$. Thus, the algorithm starts at depth 0 and reaches a maximum depth l , meaning that the largest set of evaluated variables is $\{x_1, \dots, x_k, x_{k+1}, \dots, x_{k+l}\}$. In other words, the depth corresponds to the current number of recursion levels.*

Let ORACLE be any function and assume that the depth of computed tame cases ranges from a minimum of B to a maximum of C . The MULTISOLVE algorithm implementing ORACLEH_C , which corresponds to the standard hybrid strategy, computes exactly q^C tame cases. Since it is an accurate oracle, we have no computed wild cases and the number of predicted wild cases is

$$1 + q + \dots + q^{C-1} = \frac{q^C - 1}{q - 1}.$$

Theorem 3.5. *The total numbers of tame and wild cases corresponding to any oracle function, are bounded above by the corresponding numbers for ORACLEH_C .*

Proof. Consider the q -tree associated with the recursive calls of the MULTISOLVE algorithm when implementing a general function ORACLE . If $B = C$, meaning that all tame cases occur at depth C , then the total number of tame and wild cases is exactly the same as for ORACLEH_C . If $B < C$, there exists a tame case at some depth $B \leq r < C$. In this case, the q -tree corresponding to ORACLE excludes the

q^{C-r} tame cases at depth C that would have originated from that node in the q -tree of ORACLEH_C . Similarly, the corresponding wild cases, whose total number is

$$1 + q + \dots + q^{C-r-1} = \frac{q^{C-r} - 1}{q - 1}$$

are also excluded from the q -tree defined by ORACLE . $\square$

When the oracle function is accurate, such as ORACLENRV in the attack on TRIVIUM [14], all computational effort is devoted, up to negligible overhead, to solving the tame cases, and the above result implies the superiority of the multistep strategy over the hybrid one. However, if ORACLENRV is not applicable, for instance in the sparse case where the number of remaining variables is insufficient to accurately predict the output status of GROEBNERSAFE computations, different features of the input polynomial systems has to be considered to design an accurate oracle.

A promising approach we are currently pursuing is the use of machine learning techniques to build a data-driven oracle for predicting the wall-clock runtime of the Block-Wiedemann procedure when applied to sparse Macaulay matrices arising in the multistep process. We adopt a permutation-invariant Deep Sets architecture [19], which processes each row independently, aggregates the resulting embeddings to enforce invariance with respect to row ordering, and combines them with coarse-grained metadata. The resulting global representation is then passed to a final regression head that outputs a scalar runtime prediction. The model is trained with mean squared error loss and evaluated through repeated cross-validation, assessing both predictive accuracy and how runtime relates to sparsity, matrix dimensions, and construction parameters. These results will be reported in a forthcoming paper.

The following is a trivial, non-predictive oracle that is always available

$$\text{ORACLET}(G, k) = \text{tame}.$$

Indeed, this constant tame function implies that all wild cases are computed ones. By definition, a perfect oracle function has the same computed tame cases as ORACLET and no computed wild cases. In other words, for a fixed choice of the parameters d_1, d_2 and τ , a perfect oracle achieves optimal MULTISOLVE performance in terms of the number of GROEBNERSAFE executions, that is, computed wild and tame cases. At the opposite end of the spectrum, the function ORACLET yields the worst performance.

We now compare the worst-case and optimal oracles, showing that the overhead introduced by computed wild cases remains moderate, especially when compared with the high complexity levels encountered in cryptography. Applying Theorem 3.3 then yields the following result.

Theorem 3.6. *Let N denote the number of GROEBNERSAFE executions in the MULTISOLVE algorithm using ORACLET , and let L be the corresponding number when using a perfect oracle function. The resulting speed-up ratio N/L is given by*

$$\frac{N}{L} = 1 + \frac{1 - 1/L}{q - 1}$$

and therefore

$$1 \leq \frac{N}{L} < \frac{q}{q - 1}.$$

In particular, for $q = 2$, we have $N/L = 2 - 1/L$ and $1 \leq N/L < 2$.

Proof. Recall that the recursive calls of `MULTISOLVE` define a q -ary tree. For the function `ORACLE_T`, these calls correspond exactly to executions of `GROEBNERSAFE`, and therefore their number N can be expressed as

$$N = \frac{qL - 1}{q - 1}$$

where L is the number of computed tame cases. This L also represents the total number of `GROEBNERSAFE` executions when using a perfect oracle. Computing the ratio N/L yields

$$\frac{N}{L} = \frac{qL - 1}{L(q - 1)} = 1 + \frac{1 - 1/L}{q - 1}$$

To establish the bounds, observe that the function

$$f(L) = 1 + \frac{1 - 1/L}{q - 1}$$

is strictly increasing for $L \geq 1$, since $f'(L) = \frac{1}{(q-1)L^2} > 0$. Hence, the minimum of f over $L \geq 1$ is attained at $L = 1$, giving

$$\min_{L \geq 1} \frac{N}{L} = f(1) = 1.$$

Moreover, since $1 - 1/L < 1$ for all finite $L > 1$, we obtain the strict upper bound

$$\frac{N}{L} = f(L) < 1 + \frac{1}{q - 1} = \frac{q}{q - 1}.$$

□

Since each `GROEBNERSAFE` call, whether corresponding to a tame or a wild case, is bounded by time τ , the total running time using `ORACLE_T` is at most a factor of $\frac{q}{q-1}$ larger than with a perfect oracle. In particular, for $q = 2$ this overhead is strictly less than 2, corresponding to at most a one-bit increase in the exponent of the complexity, while for large field sizes q it becomes negligible as $N/L \rightarrow 1$.

Note that choosing a constant wild oracle function reduces `MULTISOLVE` to a recursive application of `GBELIMLIN`. For $d_1 = 0$, this corresponds to brute-force enumeration of all variable assignments over the base finite field $\mathbb{F}$.

From a practical perspective, by applying `ORACLE_T` to a suitable test set of ideals $J(i_1, \dots, i_k)$, we can estimate the minimal and maximal depths for the computed tame cases, denoted by B and C respectively, for other similar ideals. Indeed, since we aim to compute $V_{\mathbb{F}}(J)$ and

$$V_{\mathbb{F}}(J) = \bigcup_{1 \leq i_1, \dots, i_k \leq q} V_{\mathbb{F}}(J(i_1, \dots, i_k))$$

it is necessary to apply `MULTISOLVE` to all ideals $J(i_1, \dots, i_k)$. The classical hybrid strategy consists of using the oracle function `ORACLEH_C`. If a perfect or, at least, an accurate oracle function is not available, a first approximation can be provided by `ORACLEH_B`. Note that using this oracle actually corresponds to executing `MULTISOLVE` with the function `ORACLE_T`, for all ideals $J(i_1, \dots, i_k, i_{k+1}, \dots, i_{k+B})$ where $1 \leq i_{k+1}, \dots, i_{k+B} \leq q$. A more refined oracle function may be derived from a detailed analysis of the distribution of wild and tame cases, for example using machine learning techniques.

4. AN OVERVIEW OF ARADI

Henceforth, we assume the base field to be the Boolean field $\mathbb{F} = \text{GF}(2)$. Aradi is a low-latency block cipher proposed by researchers at the United States National Security Agency (NSA), introduced by Greene et al. [11], to support secure memory encryption applications.

It operates on 128-bit blocks and uses a 256-bit key. The cipher follows a substitution-permutation network (SPN) design. Each round consists of three steps: key addition, a nonlinear substitution layer π , and a linear diffusion layer Λ_i ($0 \leq i \leq 3$). The full encryption process consists of 16 rounds followed by a final key addition, and is expressed as the following composition of functions:

$$\tau_{RK^{16}} \circ \bigcirc_{i=0}^{15} (\Lambda_{i \bmod 4} \circ \pi \circ \tau_{RK^i}),$$

where τ_{RK^i} denotes XOR with the i -th round key RK^i and composition $\circ$ is read from right to left.

The internal state is represented by four 32-bit words (W, X, Y, Z) .

4.1. Substitution Layer π . The substitution layer introduces nonlinearity via bitwise Toffoli gates across the word-level structure. The transformation π operates on three words at a time, following the update rule $(a, b, c) \mapsto (a, b, c \oplus (a \wedge b))$. Note that we use standard Boolean notation, where $\oplus$ denotes the XOR operation, i.e., addition in $\mathbb{F} = \text{GF}(2)$, and $\wedge$ denotes the AND operation, corresponding to multiplication in $\mathbb{F}$. Applied to the full state, π updates the words as:

$$\begin{aligned} X &\leftarrow X \oplus (W \wedge Y), \\ Z &\leftarrow Z \oplus (X \wedge Y), \\ Y &\leftarrow Y \oplus (W \wedge Z), \\ W &\leftarrow W \oplus (X \wedge Z). \end{aligned}$$

This structure can be interpreted as the parallel application of 32 instances of a 4-bit S-box, each defined by a cascade of Toffoli gates, applied independently to each bit position across the words.

4.2. Linear Layer Λ_i . The linear layer enhances diffusion and is determined by the round index modulo 4. A linear transformation L_i is applied independently to each of the four 32-bit state words:

$$\Lambda_i(W, X, Y, Z) = (L_i(W), L_i(X), L_i(Y), L_i(Z)).$$

Each map L_i is an involutive function defined on 32-bit words, where each word is viewed as a pair of 16-bit halves (u, l) . Namely, the transformation L_i is defined as follows:

$$(u, l) \mapsto \left(u \oplus S_{16}^{a_i}(u) \oplus S_{16}^{c_i}(l), l \oplus S_{16}^{a_i}(l) \oplus S_{16}^{b_i}(u) \right),$$

where S_{16}^m denotes a left circular shift by m bits on a 16-bit word. The constants (a_i, b_i, c_i) are determined based on the round index $i \bmod 4$, as specified in Table 1.

4.3. Key Addition. Each round begins with XORing a 128-bit round key to the current state. That is,

$$(W, X, Y, Z) \leftarrow (W \oplus RK_0^i, X \oplus RK_1^i, Y \oplus RK_2^i, Z \oplus RK_3^i),$$

where $(RK_0^i, RK_1^i, RK_2^i, RK_3^i)$ are the four 32-bit words that constitute the i -th round key, as defined in the key schedule depending on the parity of i .

TABLE 1. Shift offsets in the linear map Λ_i

i	a_i	b_i	c_i
0	11	8	14
1	10	9	11
2	9	4	14
3	8	9	7

4.4. Key Schedule. At step $i = 0$, the 256-bit master key is divided into eight 32-bit registers, denoted as $(K_0^0, K_1^0, \dots, K_7^0)$.

At each step $i \geq 0$, the round key RK^i is defined as follows:

$$RK^i = \begin{cases} K_0^i \| K_1^i \| K_2^i \| K_3^i & \text{if } i \text{ is even,} \\ K_4^i \| K_5^i \| K_6^i \| K_7^i & \text{if } i \text{ is odd.} \end{cases}$$

The state $(K_0^{i+1}, \dots, K_7^{i+1})$ is derived from $(K_0^i, \dots, K_7^i)$ by performing the following operations:

- First, apply the linear maps M_0 and M_1 to two pairs of registers. These are invertible linear transformations over 64 bits, acting on two 32-bit words (a, b) , and are defined as follows:

$$M_0(a, b) = (S_{32}^1(a) \oplus b, S_{32}^3(b) \oplus S_{32}^1(a) \oplus b),$$

$$M_1(a, b) = (S_{32}^9(a) \oplus b, S_{32}^{28}(b) \oplus S_{32}^9(a) \oplus b),$$

where S_{32}^m denotes the left circular shift of a 32-bit word by m positions.

- If i is even, apply M_0 to the register pairs (K_0^i, K_1^i) and (K_4^i, K_5^i) .
- If i is odd, apply M_1 to the register pairs (K_2^i, K_3^i) and (K_6^i, K_7^i) .
- Next, apply a word-level permutation to the updated state:

$$P_0 : (K_1 \leftrightarrow K_2), (K_5 \leftrightarrow K_6), \text{ for even } i,$$

$$P_1 : (K_1 \leftrightarrow K_4), (K_3 \leftrightarrow K_6), \text{ for odd } i.$$

- Finally, a round-dependent counter is XORed into register K_7 to eliminate structural symmetries across rounds.

5. SYMBOLIC MODELING OF THE ARADI CIPHER

In this section, we present a complete symbolic model of the ARADI block cipher over the Boolean field $\mathbb{F} = \text{GF}(2)$. The 128-bit plaintext and the 256-bit master key are represented as symbolic variables, while each round key bit is expressed as a linear polynomial in the master key variables, in accordance with the key schedule algorithm.

At the beginning of the first round, the 128-bit internal state, consisting of four 32-bit words, is initialized directly from the plaintext variables and updated by XORing with the 128-bit round key. This results in an internal state described by 128 linear polynomials in the plaintext and master key variables.

The updated state proceeds through the nonlinear S-box layer, where we introduce 128 new symbolic variables to represent the outputs of the 32 parallel 4-bit S-boxes. Each S-box is constrained by 21 quadratic polynomials relating its four input polynomials to its four output variables.

Following the S-box layer, the linear diffusion layer is represented by 128 linear polynomials, each expressing an output bit as a linear combination of the 128 S-box output variables.

In subsequent rounds, the internal state is updated by XORing the round key with the output of the previous round's diffusion layer. Each round repeats the modeling of the S-box and diffusion layers in the same way: the S-box layer introduces new symbolic output variables constrained by quadratic polynomials, and the diffusion layer represents its output bits as linear polynomials in the S-box output variables.

This cycle of key addition, nonlinear substitution, and linear diffusion repeats for all 16 rounds. At the end of the 16th round, the final 128 linear polynomials directly represent the ciphertext bits. Together with the quadratic polynomials generated in each round, they form a complete symbolic representation of the cipher's algebraic structure. Substituting the known plaintext values and equating the final linear polynomials to the known ciphertext bits yields a fully instantiated system of quadratic and linear equations in the master key variables and the intermediate variables introduced at each S-box layer.

The multistep solving strategy described in the previous sections is applied to the above polynomial system and the corresponding Boolean field equations, as a tool for algebraic cryptanalysis. The following notation and setup provide the basis for the formal round-by-round description of the symbolic model.

Notation and Setup. We work over the Boolean field $\mathbb{F} = \text{GF}(2)$ and consider the multivariate polynomial algebra

$$R = \mathbb{F}[k_0, \dots, k_{255}, W^0, X^0, Y^0, Z^0, S_W^i, S_X^i, S_Y^i, S_Z^i \ (1 \leq i \leq 16)]$$

where:

- $k_0, \dots, k_{255}$ are the 256 master key variables;
- W^0, X^0, Y^0, Z^0 denote the four 32-bit words of the plaintext;
- $S_W^i, S_X^i, S_Y^i, S_Z^i$ ($1 \leq i \leq 16$) denote the 4 S-box layer output words of 32 symbolic variables each, introduced in round i .

For each round i , the 128-bit round key is denoted by $RK^i = (K_W^i, K_X^i, K_Y^i, K_Z^i)$, where each K_T^i ($T = W, X, Y, Z$) is a vector of 32 linear polynomials in the 256 master key variables:

$$\begin{aligned} K_W^i &= (k_{w,0}^i, \dots, k_{w,31}^i), & K_X^i &= (k_{x,0}^i, \dots, k_{x,31}^i), \\ K_Y^i &= (k_{y,0}^i, \dots, k_{y,31}^i), & K_Z^i &= (k_{z,0}^i, \dots, k_{z,31}^i). \end{aligned}$$

To describe the round-by-round evolution of the internal state, we denote by W^i, X^i, Y^i, Z^i the symbolic 32-bit words at the input of round i , represented as polynomials in R .

We now describe the symbolic modeling of each round's operations within the polynomial algebra R .

Round Key Addition. At the beginning of round i , one has that the internal state (W^i, X^i, Y^i, Z^i) , each a 32-bit word represented as polynomials in R , is updated by XORing with the corresponding round key $(K_W^i, K_X^i, K_Y^i, K_Z^i)$. Explicitly, for each bit index $0 \leq j \leq 31$, it holds

$$\bar{w}_j^i = w_j^i + k_{w,j}^i, \bar{x}_j^i = x_j^i + k_{x,j}^i, \bar{y}_j^i = y_j^i + k_{y,j}^i, \bar{z}_j^i = z_j^i + k_{z,j}^i.$$

The resulting polynomials $\bar{w}_j^i, \bar{x}_j^i, \bar{y}_j^i, \bar{z}_j^i \in R$ form the input to the nonlinear substitution layer.

Nonlinear Substitution. For each index $0 \leq j \leq 31$ in round i , the 4-bit input to the S-box layer is

$$(x_0, x_1, x_2, x_3) = (\bar{w}_j^i, \bar{x}_j^i, \bar{y}_j^i, \bar{z}_j^i),$$

and the 4-bit output is

$$(y_0, y_1, y_2, y_3) = (s_{w,j}^i, s_{x,j}^i, s_{y,j}^i, s_{z,j}^i).$$

The nonlinear substitution is modeled, for each S-box, by 21 quadratic polynomials in the polynomial algebra R , which implicitly define the S-box transformation $\text{Sbox}(x_0, x_1, x_2, x_3) = (y_0, y_1, y_2, y_3)$, namely

$$\begin{aligned} & x_0x_2 + x_1x_2 + x_3 + y_3, \quad x_0x_1 + x_0x_2 + x_0y_1, \quad x_0x_2 + x_0y_2 + x_0y_3, \quad x_0x_2 + x_0x_3 \\ & + x_1y_1 + x_0y_2 + y_1, \quad x_0x_3 + x_1x_3 + x_0y_0 + x_0y_2 + x_1y_2 + y_0, \quad x_0x_2 + x_0x_3 + x_1x_3 \\ & + x_0y_2 + x_1y_3 + x_3 + y_3, \quad x_0x_2 + x_1 + y_1, \quad x_0x_2 + x_0y_2 + x_2 + y_2, \quad x_2y_1 + x_3 + y_3, \\ & x_0x_2 + x_0x_3 + x_1x_3 + x_0y_2 + x_2y_2 + x_0 + x_2 + x_3 + y_0 + y_3, \quad x_2x_3 + x_2y_3 + x_3 + y_3, \\ & x_0x_2 + x_0x_3 + x_2y_0 + x_3y_0 + x_0 + y_0, \quad x_3y_1 + x_0 + x_3 + y_0 + y_3, \quad x_0x_1 + x_0x_2 + x_2x_3 \\ & + x_1y_0 + x_0y_2 + x_3y_2 + x_0 + y_0, \quad x_0x_2 + x_2y_0 + x_3y_3 + y_3, \quad x_0x_1 + x_0x_2 + y_0y_1 \\ & + x_0 + y_0, \quad x_0x_2 + x_0y_0 + x_2y_0 + x_0y_2 + y_0y_2 + x_0, \quad x_0x_2 + x_0y_2 + y_0y_3 + x_0 + y_0, \\ & x_0y_0 + y_1y_2 + x_0 + x_3 + y_3, \quad y_1y_3 + x_0 + y_0, \quad x_0x_2 + x_2x_3 + x_0y_2 + y_2y_3 + x_3 + y_3. \end{aligned}$$

Altogether, the 32 parallel S-boxes contribute a total of 672 quadratic polynomials in R for round i .

Word-wise Linear Transformation. The linear diffusion layer applies the transformation $L_{(a_i, b_i, c_i)}$ to each 32-bit word produced by the S-box layer. Consider, for example, one such word, say $S_W^i = (s_{w,0}^i, \dots, s_{w,31}^i)$, and split it into two 16-bit symbolic halves:

$$l^i = (s_{w,0}^i, \dots, s_{w,15}^i), \quad r^i = (s_{w,16}^i, \dots, s_{w,31}^i).$$

The transformation $L_{(a_i, b_i, c_i)}(S_W^i)$ produces new linear polynomials in R with the updated halves $(l_0^i, \dots, l_{15}^i)$ and $(r_0^i, \dots, r_{15}^i)$ defined for each $0 \leq j \leq 15$ by:

$$\begin{aligned} l_j^i &= l_j^i + l_{(j+a_i) \bmod 16}^i + r_{(j+c_i) \bmod 16}^i, \\ r_j &= r_j^i + r_{(j+a_i) \bmod 16}^i + l_{(j+b_i) \bmod 16}^i. \end{aligned}$$

where the addition in subscripts is ordinary integer addition, while all other additions are in the polynomial algebra R over the base field $\mathbb{F} = \text{GF}(2)$. The constants (a_i, b_i, c_i) are specified in Table 1.

This transformation $L_{(a_i, b_i, c_i)}$ is applied independently to the four symbolic S-box output words, yielding the updated four 32-bit symbolic words of the internal state for the next round:

$$\begin{aligned} W^{i+1} &= L_{(a_i, b_i, c_i)}(S_W^i), & X^{i+1} &= L_{(a_i, b_i, c_i)}(S_X^i), \\ Y^{i+1} &= L_{(a_i, b_i, c_i)}(S_Y^i), & Z^{i+1} &= L_{(a_i, b_i, c_i)}(S_Z^i). \end{aligned}$$

These updated words consist of linear polynomials in the algebra R , and no new symbolic variables are introduced at this step.

Each round of the ARADI cipher is symbolically modeled by quadratic polynomials from the S-box layer and linear polynomials from the key addition and diffusion

layers, all defined over the multivariate polynomial algebra R . Since the output of the diffusion layer becomes the input to the next round's key addition, and the resulting expressions feed into the nonlinear S-boxes, the full symbolic model of the 16-round cipher comprises 16×672 quadratic polynomials from the S-box layers, along with 128 linear polynomials from the final diffusion layer. Altogether, the system consists of 10,880 multivariate polynomials in R , involving 2,432 variables: 256 master key bits, 128 plaintext bits, and 128×16 intermediate S-box output bits. These polynomials collectively define an implicit algebraic representation of the ARADI cipher.

For any known plaintext-ciphertext pair, substituting the known values results in a fully instantiated system of polynomial equations in the master key variables and the intermediate S-box output variables. Along with the Boolean field equation ideal $\mathcal{F}$ corresponding to the polynomial algebra R , this system forms the input for the multistep polynomial system solver used in a key-recovery attack, which we discuss in the following sections.

We note that the final key addition layer of the ARADI encryption is not considered in this symbolic representation, as its linear nature does not affect the algebraic complexity relevant to our key-recovery attack.

6. AN ALGEBRAIC ATTACK TO ARADI BASED ON THE MULTISTEP STRATEGY

This section explains how the multistep solving strategy can be applied to perform an algebraic attack on the full-round ARADI cipher. Given a known plaintext-ciphertext pair, the plaintext variables in the system are assigned fixed values. The 128 linear polynomials produced by the final round's linear layer are then equated to the corresponding ciphertext bits, resulting in a system of multivariate polynomial equations.

The system involves 256 master key variables and $2048 = 16 \times 128$ auxiliary variables, representing the outputs of the 32 parallel 4-bit S-boxes across 16 rounds. In total, it comprises $16 \times 672 = 10,752$ quadratic equations arising from the S-box layers, along with 128 final linear equations, yielding a total of 10,880 equations in the multivariate polynomial algebra

$$R = \mathbb{F}[k_0, \dots, k_{255}, W^0, X^0, Y^0, Z^0, S_W^i, S_X^i, S_Y^i, S_Z^i \ (1 \leq i \leq 16)].$$

where $\mathbb{F} = \text{GF}(2)$. All computations for solving the system are performed modulo the Boolean field equation ideal $\mathcal{F} \subset R$.

Our goal is to recover the values of the unknown key bits $k_0, \dots, k_{255}$ by solving this system. Direct Gröbner basis computation on the full system is computationally infeasible. In particular, matrix-based algorithms such as F4, F5 or XL [7, 8, 9] suffer from exponential growth in the number of monomials and matrix dimensions. This quickly leads to memory exhaustion, even when using sparse matrix techniques.

To address this, the classical hybrid solving strategy is often employed in algebraic cryptanalysis. However, the number of variables that must be exhaustively guessed is typically large in order to make all resulting polynomial systems feasibly solvable. Moreover, this number is generally determined empirically, requiring numerous attempts with different number of variables and guesses to identify a workable configuration.

The multistep strategy offers a more automated algorithm that, starting from a reasonable initial set of guessed variables, explores the algebraic structure of the polynomial system and extends this set only when strictly necessary.

The multistep solving strategy, namely the MULTISOLVE algorithm, has been implemented in the computer algebra system MAGMA [16], utilizing its built-in Gröbner basis computation engine. Given the high sparsity of the polynomial systems involved in the algebraic attack on ARADI, we make use of the sparse variant of the F4 algorithm.

The multivariate polynomial system representing the full 16-round attack on ARADI is derived according to the symbolic modeling described in Section 5.

To evaluate the effectiveness of the multistep strategy, we performed a key-recovery attack using a random plaintext-ciphertext pair derived from a randomly generated master key. Each experiment begins by generating a random master key, selecting a random plaintext, and computing the corresponding ciphertext. Subsequently, a random initial assignment is generated for a subset of the master key variables. Using this information, the associated polynomial system is instantiated, and the MULTISOLVE algorithm is applied to attempt its resolution. Since the initial guesses for some master key bits are random, and therefore likely incorrect, all the polynomial systems generated along the multistep strategy are inherently inconsistent.

To construct the initial subset of variables to be guessed, the 256 key variables are first partitioned into 8 blocks of 32 variables each, namely

$$B_l = \{k_{32l+i} \mid 1 \leq i \leq 32\} \quad (0 \leq l \leq 7).$$

Various permutations of these blocks are then generated. For each permutation, a subset of k variables is selected by taking the first k variables from the concatenated sequence of the permuted blocks. By empirical analysis, the fastest choice appears to be the reverse permutation

$$B_7 \cup B_6 \cup \dots \cup B_1 \cup B_0.$$

We developed a comprehensive and versatile implementation of the MULTISOLVE algorithm within the MAGMA computer algebra system. The implementation supports a range of oracle functions, allowing for flexible experimentation with different heuristics and decision strategies throughout the solving process.

Through a series of experiments on ARADI, we identified effective parameter settings for MULTISOLVE in the context of this attack: specifically, $d_1 = 0$, which disables GBELIMLIN, and $d_2 = 2$ for GROEBNERSAFE. This choice is motivated by the fact that the input polynomials are quadratic, which enables GROEBNERSAFE to be applied efficiently at this degree while still producing complete Gröbner bases, thus rendering GBELIMLIN unnecessary.

For wild cases, we set a time limit of $\tau = 60$ seconds. The oracle function employed in MULTISOLVE is ORACLE_T. After conducting several tests with an initial number of variables set to 240, it became clear that tame cases always occur when the number of guessed key variables is 251 or 252. To obtain the classic hybrid strategy, it was therefore necessary to begin with 252 evaluations. Our experiments also confirmed that assigning correct values to 251 or 252 key variables is sufficient to recover the master key.

A total of 2^{16} experiments were performed, each using a distinct random master key, plaintext, and partial assignment of key bits. For MULTISOLVE, 99.98% of the

cases were tame with 251 assignments. The cost of identifying the remaining 0.02% of wild cases via timeout and resolving them with 252 key variable evaluations was therefore negligible.

We report here the average runtimes of GROEBNERSAFE in solving tame cases, along with the corresponding standard deviations to highlight the stability of the results. All timings are given in seconds. All computations were performed on a Linux-based system running Ubuntu 22.04 LTS, equipped with an AMD EPYC 7763 64-core processor and x86_64 architecture. For Gröbner basis computations, we used the sparse F4 algorithm as implemented in MAGMA version 2.27.5.

TABLE 2. GB Average Runtime for Tame Cases

# Guessed Vars	Solving Time (s)	Std. Dev. (s)
251	34.629	1.892
252	24.957	1.756

Since $\log_2(34.62) = 5.11$ and $\log_2(24.95) = 4.64$, the overall complexity of the multistep strategy is approximately 2^{256} seconds, while that of the classical hybrid strategy is around $2^{256.5}$ seconds. Given that a single encryption in a brute force attack takes significantly less than one second, we cannot claim that ARADI is broken. However, our attack has revealed some structural features of the symbolic model.

Indeed, our experiments show that assigning 251 key variables leads to a substantial reduction in the complexity of solving the derived quadratic polynomial systems. Only a small fraction of the instances require 252 assignments, and all systems are solved in approximately 30 seconds with a solving degree of two, equal to the input degree. This behavior is likely due to the extreme sparsity of the corresponding Macaulay matrices and the fact that these polynomial systems deviate significantly from the semi-regular case. Notably, the systems involve around 2000 variables.

We also experimented with various combinations of the parameters d_1, d_2 and τ , but none of these configurations enabled us to obtain tame cases with only 250 key variable evaluations. In other words, fixing 251 key variables in ARADI’s symbolic model results in a marked drop in computational effort.

7. CONCLUSIONS AND FURTHER DIRECTIONS

In this paper, we propose a new version of the multistep solving strategy, based on a recursive Depth-First Search implementation and on the introduction of an “oracle function” intended to predict whether a polynomial system requires further simplification through variable evaluations.

This approach enables a simple yet effective implementation of the MULTISOLVE algorithm and supports a complexity analysis based on tree structures. Furthermore, it provides a unified framework that generalizes various solving strategies, including the one used in our attack on the TRIVIUM cipher and the classic hybrid approach, into a single algorithm, where only the oracle function needs to be adapted. Thanks to this work, we were able to perform an algebraic attack on the recent block cipher ARADI, which revealed some aspects of its symbolic modeling.

As a further direction for this line of research, we suggest developing oracle functions based on machine learning algorithms, trained on the ideals and corresponding

Macaulay matrices generated by MULTISOLVE. A preliminary experimental study is currently underway. Another promising direction lies in the development of a parallel implementation of the MULTISOLVE algorithm, as well as in its extension to polynomial systems admitting multiple solutions, which could be leveraged in discrete optimization tasks.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their careful reading of the manuscript and for their constructive comments, which enhanced the clarity and readability of the work.

REFERENCES

- [1] Avanzi, R.; Dunkelman, O.; Ghosh, S., A Note on ARADI and LLAMA. Cryptology ePrint Archive, Paper 2024/1328, 2024. <https://eprint.iacr.org/2024/1328>
- [2] Bettale, L.; Faugère, J.-C.; Perret, L., Hybrid approach for solving multivariate systems over finite fields. *J. Math. Cryptol.*, 3 (2009), no. 3, 177–197.
- [3] Bettale, L.; Faugère, J.-C.; Perret, L., Solving Polynomial Systems over Finite Fields: Improved Analysis of the Hybrid Approach. International Symposium on Symbolic and Algebraic Computation - ISSAC 2012, 67–74, 2012.
- [4] Bellini, E.; Rachidi, M.; Rohit, R.; Tiwari, S.K., Mind the Composition of Toffoli Gates: Structural Algebraic Distinguishers of ARADI. Cryptology ePrint Archive, Paper 2024/1559, 2024. <https://eprint.iacr.org/2024/1559>
- [5] Bellini, E.; Rachidi, M.; Rohit, R.; Tiwari, S.K., On the Structural Properties of Toffoli Gate Composition in ARADI: Implications for Algebraic Distinguishers. to appear in Proceedings of the Applied Cryptography and Network Security (ACNS) 2025, Springer, 2025.
- [6] Bellini, E.; Formenti, M.; Gérault, D.; Grados, J.; Hambitzer, A.; Huang, Y.J.; Huynh, P.; Rachidi, M.; Rohit, R.; Tiwari, S.K., CLAASping ARADI: Automated Analysis of the ARADI Block Cipher. Progress in Cryptology – INDOCRYPT 2024, pp. 90–113, Springer Nature Switzerland, 2025. https://doi.org/10.1007/978-3-031-80311-6_5
- [7] Courtois, N.; Klimov, A.; Patarin, J.; Shamir, A., Efficient algorithms for solving overdefined systems of multivariate polynomial equations. Advances in Cryptology - EUROCRYPT 2000, Lecture Notes in Comput. Sci., 1807, 392–407, Springer, Berlin, 2000.
- [8] Faugère, J.C., A new efficient algorithm for computing Gröbner bases (F4). Effective Methods in Algebraic Geometry - MEGA 1998. *J. Pure Appl. Algebra*, 139 (1999), no. 1-3, 61–88.
- [9] Faugère, J.C., A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation, 75–83. ACM, New York, 2002.
- [10] Gerdts, V.; La Scala, R., Noetherian quotients of the algebra of partial difference polynomials and Gröbner bases of symmetric ideals *J. Algebra*, 423 (2015), 1233–1261.
- [11] Greene, P.; Motley, M.; Weeks, B., ARADI and LLAMA: Low-Latency Cryptography for Memory Encryption. Cryptology ePrint Archive, Paper 2024/1240, 2024. <https://eprint.iacr.org/2024/1240>
- [12] Kim, S.; Kim, I.; Lee, D.; Hong, D.; Sung, J.; Hong, S., Byte-wise Equal Property of ARADI. Cryptology ePrint Archive, Paper 2024/1772, 2024. <https://eprint.iacr.org/2024/1772>
- [13] La Scala, R.; Polese, S.; Tiwari, S.K.; Visconti, A., An algebraic attack to the Bluetooth stream cipher E0. *Finite Fields Appl.*, 84 (2022), Paper No. 102102, 29 pp.
- [14] La Scala, R.; Pintore, F.; Tiwari, S.K.; Visconti, A., A multistep strategy for polynomial system solving over finite fields and a new algebraic attack on the stream cipher Trivium. *Finite Fields Appl.*, 98 (2024), Paper No. 102452, 33 pp.
- [15] La Scala, R.; Tiwari, S. K., Stream/block ciphers, difference equations and algebraic attacks. *J. Symbolic Comput.*, 109 (2022), 177–198.
- [16] Bosma, W.; Cannon, J.J.; Playoust, C., The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24 (1997), 235–265.
- [17] Massacci, F.; Marraro, L., Logical cryptanalysis as a SAT problem. *J. Automat. Reason.*, 24 (2000), no. 1-2, 165–203.

- [18] Ramos-Calderer, S., Bravo-Prieto, C.; Lin, R.; Bellini, E.; Manzano, M.; Aaraj, N.; Latorre, J.I., Solving systems of Boolean multivariate equations with quantum annealing, *Phys. Rev. Res.*, 4, 013096, 2022.
- [19] Zaheer, M.; Kottur, S.; Ravanbakhsh, S.; Poczos, B.; Salakhutdinov, R.R.; Smola, A.J., Deep Sets. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS) 2017, vol. 30, pp. 3391–3401, Curran Associates, Inc., 2017.

* DIPARTIMENTO DI MATEMATICA, UNIVERSITÀ DEGLI STUDI DI BARI “ALDO MORO”, VIA ORABONA 4, 70125 BARI, ITALY

Email address: roberto.lascalea@uniba.it

** CRYPTOGRAPHY RESEARCH CENTRE, TECHNOLOGY INNOVATION INSTITUTE, ABU DHABI, UNITED ARAB EMIRATES

Email address: sharwan.tiwari@tii.ae